Beyond Driscoll–Healy: Drop-in Quadrature Upgrades for EquiformerV2's S^2 Activation

Anonymous Authors

Abstract

EquiformerV2 and other recent SO(3)-equivariant atomistic networks implement a pointwise S^2 activation by mapping spherical-harmonic coefficients to a discrete grid, applying a scalar nonlinearity, and integrating back. The default grid, inherited from `e3nn`, uses Driscoll–Healy (DH) equiangular quadrature. We argue that this default is not the best choice for EquiformerV2, and that the choice of *quadrature method* (rather than the activation function or the sampling pattern, the targets of concurrent work) is itself a useful lever for the speed/accuracy trade-off. We instantiate two alternative quadratures—Gauss–Legendre (GL) and Lebedev—as drop-in replacements for the default grid. On the fairchem-default 12-layer EquiformerV2 ($\ell_{\max}$=6), GL match-DH reaches the same S^2-activation equivariance error as the standard DH-2× oversampling at 113.17 ± 2.25 ms per forward (versus 164.66 ± 1.62 ms for DH-2×): a saving of 51.5 ± 2.36 ms (31.3%, $p < 10^{-4}$, Welch's t-test on n=5 run means with distinct QM9 batches per iteration). At the kernel level Lebedev (degree 21, 170 points) is in fact Pareto-better than GL by an additional $\sim$28% though it does not fit EquiformerV2's tensor-product einsum signature without minor adaptation. The swap is safe in inference: on five 50-epoch DH-trained QM9 checkpoints the test MAE shifts by ≤ 0.01 eV and the per-layer ℓ=0 invariance profile is preserved across all grids. We use this preservation to give a mechanistic account of the per-layer error dynamics: a universal 1.5–16× jump from layer 0 to layer 1 followed by a plateau, set by the cross-degree coupling factor of the SO(2) convolution and the smoothness of the activation, with the absolute floor fixed by the training-time grid.

1 Introduction

The recent line of SO(3)-equivariant graph networks for molecular and materials property prediction Zitnick et al. [2022], Passaro and Zitnick [2023], Liao et al. [2024] represents per-node features as functions on the sphere, expanded in spherical harmonics (SH) up to a band limit $\ell_{\max}$. The dominant nonlinearity in this design is the S^2 *activation*: it projects SH coefficients onto a finite grid on S^2, applies a scalar activation pointwise, and integrates back via quadrature. This operator is exactly equivariant only when the quadrature integrates the relevant products of band-limited functions exactly; otherwise its equivariance violation propagates through the rest of the network.

The size of this violation is well known to the community. The EquiformerV2 paper itself notes that "sampling-induced equivariance error remains an open challenge" Liao et al. [2024], and the fairchem issue tracker (`FAIR-Chem/fairchem#1068`, March 2025) records users observing equivariance error of order $\sim 10^{-2}$ at the default grid and only $\sim 10^{-4}$ at 1024^2 grid points—a prohibitive cost fairchem community [2025]. Two recent papers attack the problem with different tools. EquiformerV3 Liao et al. [2026] replaces the S^2 activation with a SwiGLU-style operator on scalars and bilinear products, achieving a 50% reduction in grid points but requiring an architectural change and retraining. The Equivariant Spherical Transformer (EST) An et al. [2025] replaces the e3nn grid with a Fibonacci

Preprint. Under review.

lattice—a heuristically more uniform sampling pattern. What none of these works changes is the *quadrature rule*.

EquiformerV2's `SO3_Grid` delegates to e3nn's `ToS2Grid`/`FromS2Grid`, which use an equiangular Driscoll–Healy (DH) grid with Kostelec–Rockmore weights. This was a defensible inheritance because DH supports an FFT-based fast SH transform, the algorithmic property around which e3nn was originally designed. EquiformerV2, however, does not use the FFT: its `SO3_Grid` precomputes `to_grid_mat` and `from_grid_mat` as dense matrices and applies them through `einsum` (§2). Once committed to dense matrix multiplication, the question collapses to "which set of nodes integrates band-limited products with the fewest points?" and the textbook answer is no longer Driscoll–Healy: Gauss–Legendre needs roughly half the latitude points of DH for matched exactness Wieczorek and Meschede [2018], Hirt et al. [2015], Driscoll and Healy [1994], and Lebedev rules can do better still for moderate-degree spherical integrands.

In this paper we treat the choice of quadrature method as an explicit hyperparameter of the S^2 activation. We expose it through a drop-in `CustomSO3Grid` module that replaces the default DH grid in any EquiformerV2 backbone (§3). We instantiate two alternatives, Gauss–Legendre and Lebedev, and characterise their behaviour at three levels: equivariance error of the bare S^2-Activation kernel, end-to-end forward wall-clock under a rigorous independent-replicate protocol (§4), and preservation of trained-model predictions when the grid is swapped post-hoc into existing checkpoints (§5). Per-layer measurements of the $\ell{=}0$ invariance error connect the two, exposing a universal two-phase pattern—a sharp jump from layer 0 to layer 1, then a plateau—whose absolute floor is set by the training-time grid.

We make four contributions. ***First***, we identify the choice of quadrature method as a separate axis of speed/accuracy improvement for EquiformerV2's S^2 activation, complementary to the architectural and sampling-pattern fixes proposed concurrently in Liao et al. [2026] and An et al. [2025], and demonstrate it with two concrete instantiations (Gauss–Legendre and Lebedev) deployed through a drop-in module. ***Second***, on the fairchem-default 12-layer EquiformerV2 we measure that GL match-DH reaches the same equivariance floor as the standard DH $2\times$ oversampling at 31.3% lower wall-clock per forward (113.17 ± 2.25 vs. 164.66 ± 1.62 ms, $p < 10^{-4}$ on $n{=}5$ independent replicates), while the GL configurations themselves are statistically indistinguishable from the DH default in wall-clock ($p > 0.4$). ***Third***, on five 50-epoch DH-trained QM9 checkpoints, swapping the grid at inference shifts test MAE by at most 0.01 eV and leaves the trained-model prediction-invariance ratio at 1.00 ± 0.02, establishing the swap as a deployment-ready drop-in for inference cost. ***Fourth***, a per-layer analysis exposes a universal 1.5–$16\times$ jump at layer $0{\to}1$ followed by a plateau, captured by a δ/c toy model in which δ is the per-S^2-Act $\ell{>}0$ perturbation and c is the cross-degree coupling factor. The shape is set by the architecture; the *absolute floor* is set by the training-time grid's *point count*, not its method—training with GL match-DH (182 points) gives a profile statistically indistinguishable from DH default (70 points), while training with the much denser DH $2\times$ (784 points) lowers the floor by $\sim 160\times$. This explains why an inference-time swap preserves predictions (the profile does not move under it) and why a trained-model equivariance gain requires training with substantially more grid points than the e3nn default; GL gives the same wall-clock as DH default at any matched grid budget, so a denser GL grid (e.g. GL $2\times$) is the natural intervention.

2 Background

A per-node feature in an SO(3)-equivariant network on S^2 is a vector $c \in \mathbb{R}^{(\ell_{\max}+1)^2}$ of spherical-harmonic coefficients indexed by (ℓ, m) with $0 \le \ell \le \ell_{\max}$ and $-\ell \le m \le \ell$. EquiformerV2 additionally crops to $|m| \le m_{\max} \le \ell_{\max}$, an SO(3)-to-SO(2) reduction introduced by eSCN Passaro and Zitnick [2023]. Given any quadrature $\{(\theta_i, \phi_i, w_i)\}_{i=1}^{N}$ on the sphere with $\sum_i w_i = 4\pi$ and a scalar function $\sigma : \mathbb{R} \to \mathbb{R}$, the S^2 activation $\mathcal{A}_\sigma$ is

$$\mathcal{A}_\sigma(c)_{\ell m} \;=\; \sum_{i=1}^{N} w_i\, \sigma\left(\sum_{\ell', m'} c_{\ell' m'}\, Y_{\ell' m'}(\theta_i, \phi_i) \right) Y_{\ell m}(\theta_i, \phi_i), \tag{1}$$

where $Y_{\ell m}$ are the real spherical harmonics. Equation (1) factorises naturally as two linear maps separated by a pointwise nonlinearity: a forward projection $c \mapsto f$ that evaluates $f(\theta_i, \phi_i) = \sum_{\ell', m'} c_{\ell' m'} Y_{\ell' m'}(\theta_i, \phi_i)$ at every grid point, the application of σ to each $f(\theta_i, \phi_i)$, and a backward

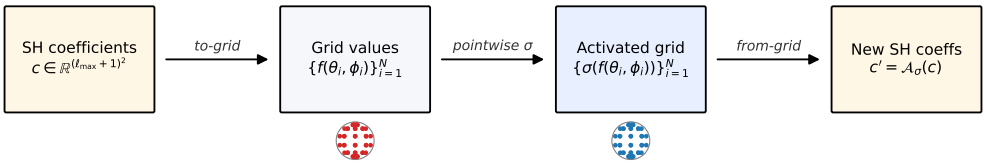

Figure 1: The S^2 activation pipeline. SH coefficients c are projected to grid values $f(\theta_i, \phi_i)$ (the *to-grid* step), the scalar activation σ is applied pointwise on S^2, and the result is integrated against $w_i\, Y_{\ell m}$ to produce new SH coefficients c' (the *from-grid* step). Only the from-grid step weights the values, and only it contributes quadrature error. Both projections use the same set of quadrature nodes $\{(\theta_i, \phi_i)\}$, illustrated for three families in Figure 2.

projection that integrates against $w_i Y_{\ell m}(\theta_i, \phi_i)$. EquiformerV2's `S2Activation.forward` implements exactly this factorisation (Figure 1), with the two linear maps stored as dense buffers $\texttt{to\_grid\_mat}_{i,(\ell m)} = Y_{\ell m}(\theta_i, \phi_i)$ and $\texttt{from\_grid\_mat}_{i,(\ell m)} = w_i\, Y_{\ell m}(\theta_i, \phi_i)$. Because the buffers are precomputed once at model construction, replacing the grid amounts to replacing these two buffers; no other code in the network changes.

We will distinguish two equivariance metrics throughout. The first is the *kernel-level* equivariance error of a single S^2 activation, which we define as the expected relative residual under random rotations and random SH coefficients. Let $\widetilde{D}(R)$ denote the Wigner-D matrix associated with a rotation $R \in \mathrm{SO}(3)$ (acting on the SH-coefficient vector by $c \mapsto \widetilde{D}(R)c$):

$$e_{\mathrm{kernel}}(\mathcal{A}_\sigma) \;=\; \mathop{\mathbb{E}}_{\substack{c\sim\mathcal{N}(0,I)\\ R\sim\mathrm{Haar}(\mathrm{SO}(3))}} \left[\frac{\left\| \mathcal{A}_\sigma\big(\widetilde{D}(R)\, c\big) \;-\; \widetilde{D}(R)\, \mathcal{A}_\sigma(c) \right\|}{\left\| \mathcal{A}_\sigma\big(\widetilde{D}(R)\, c\big) \right\|} \right]. \tag{2}$$

In practice we estimate (2) as a Monte Carlo average over 50 independent (c, R) pairs (5 rotations $\times$ 10 random inputs in the experiments below). This metric depends only on the grid and the activation σ; it does not require a trained model. The second is the *trained-model prediction invariance* of the full network's output y on a molecular input x:

$$e_{\mathrm{pred}} \;=\; \mathop{\mathbb{E}}_{\substack{x\sim\mathcal{D}_{\mathrm{test}}\\ R\sim\mathrm{Haar}(\mathrm{SO}(3))}} \left[\frac{\big| y(x) - y(R\cdot x) \big|}{\big| y(x) \big|} \right], \tag{3}$$

where $\mathcal{D}_{\mathrm{test}}$ is the held-out test set and $R \cdot x$ denotes the rotation of atom positions by R. This metric depends on the trained weights, the architecture, and *every* S^2 activation in the network; we estimate it as an average over 10 rotations $\times$ 20 test batches. The two metrics agree qualitatively but differ by orders of magnitude in absolute scale ($e_{\mathrm{kernel}} \sim 10^{-1}$ vs. $e_{\mathrm{pred}} \sim 10^{-5}$ in our setting); we always state explicitly which one each number reports.

The choice of quadrature in (1) amounts to choosing $\{(\theta_i, \phi_i, w_i)\}_{i=1}^N$, and is the central design question this paper addresses (Section 3).

3 Method: `CustomSO3Grid`

We consider three quadrature families on S^2 (Figure 2). *Driscoll–Healy* uses an equiangular tensor-product grid $\theta_j = (j + \frac{1}{2})\pi/N_\theta$, $\phi_k = 2\pi k/N_\phi$, with $N_\theta = 2(\ell_{\max} + 1)$ latitude points and $N_\phi \geq 2\ell_{\max} + 1$ longitude points Driscoll and Healy [1994]. *Gauss–Legendre* places θ_j at the roots of the Legendre polynomial $P_{N_\theta}(\cos\theta_j) = 0$ with companion weights, which is exact for band-limited products of degree $\leq 2\ell_{\max}$ using only $N_\theta = \ell_{\max} + 1$ latitude points—a factor of two fewer than DH Wieczorek and Meschede [2018]; ϕ is again uniform. *Lebedev* rules Lebedev [1976] give a non-tensor-product set of N nodes on the sphere designed to be exact for all spherical polynomials up to a given degree with as few points as possible: degree 21 with 170 points is the smallest rule exact at $\ell_{\max}$=6, far below DH's $14 \times 13 = 182$ points or DH 2×'s $28 \times 28 = 784$ points.

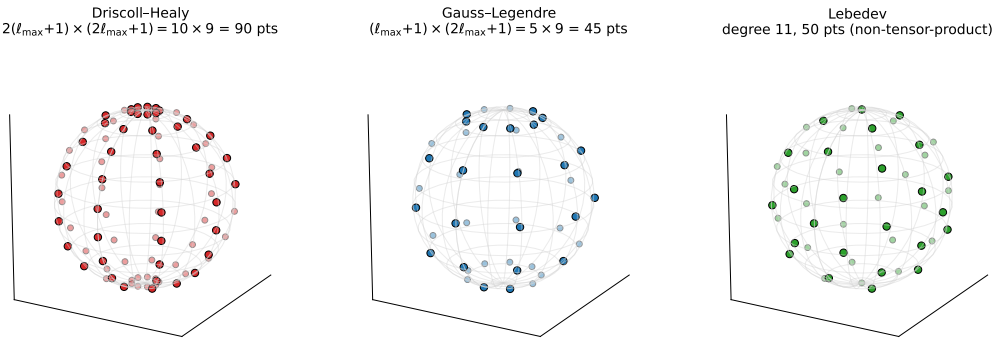

Figure 2: Three quadrature node layouts on S^2 for $\ell_{\max} = 4$. Driscoll–Healy (left) uses an equiangular tensor-product grid with $2(\ell_{\max}+1)$ latitude rings, twice what Gauss–Legendre (middle) needs for the same exactness. Lebedev (right) avoids the tensor-product structure entirely, distributing roughly the same number of points more uniformly over the sphere. All three integrate spherical polynomials of degree $\leq 2\ell_{\max}$ exactly. Replacing one for another changes only the buffers `to_grid_mat` and `from_grid_mat`; the rest of the Figure 1 pipeline is invariant.

We expose all three through a drop-in module `CustomSO3Grid` that replaces fairchem's `SO3_Grid`. It uses the same `get_to_grid_mat` / `get_from_grid_mat` interface, with matrices of identical shape signature; a `patch_so3_grid()` helper swaps every grid in `model.backbone.SO3_grid` after model construction, so existing pretrained weights remain valid.

3.1 Constructing the to/from-grid matrices

For the tensor-product cases (DH and GL) the grid factorises as $\{(\theta_j, \phi_k, w_j^{(\theta)} w_k^{(\phi)})\}$ with $1 \leq j \leq N_\theta$ latitude points and $1 \leq k \leq N_\phi$ longitude points. The longitude weights are uniform, $w_k^{(\phi)} = 2\pi/N_\phi$. The latitude nodes and weights differ:

- *Driscoll–Healy:* $\theta_j^{\mathrm{DH}} = (j - \frac{1}{2})\pi/N_\theta$ (uniform in θ), with weights given by the Kostelec–Rockmore expression $w_j^{\mathrm{DH}} = (2/N_\theta)\sin\theta_j \sum_{r=0}^{N_\theta/2-1} \frac{1}{2r+1}\sin\big((2r+1)\theta_j\big)$ Kostelec and Rockmore [2008], and $N_\theta = 2(\ell_{\max}+1)$.
- *Gauss–Legendre:* θ_j^{GL} are the roots of the Legendre polynomial $P_{N_\theta}(\cos\theta_j^{\mathrm{GL}}) = 0$ on $(0, \pi)$, with companion weights $w_j^{\mathrm{GL}} = 2/\big[(1-\cos^2\theta_j)\,(P'_{N_\theta}(\cos\theta_j))^2\big]$, and $N_\theta = \ell_{\max}+1$ suffices for exactness on band-limited products of degree $2\ell_{\max}$.

Both yield a flat list of $N = N_\theta N_\phi$ nodes (θ_i, ϕ_i) with weights $w_i = w_j^{(\theta)} w_k^{(\phi)}$. We assemble the buffers as $\texttt{to\_grid\_mat}_{i,(\ell m)} = Y_{\ell m}(\theta_i, \phi_i)$ and $\texttt{from\_grid\_mat}_{i,(\ell m)} = w_i\, Y_{\ell m}(\theta_i, \phi_i)$, using e3nn's real spherical harmonics `o3.spherical_harmonics(..., normalization='integral')` and applying the same $m_{\max}$-cropping and rescaling as fairchem's `SO3_Grid` (`so3.py:537-575`). Reshaping into the $[N_\theta, N_\phi, n_{\mathrm{coeffs}}]$ layout that `S2Activation.forward` expects is a no-op for the tensor-product cases.

For Lebedev rules the grid does not factorise. The nodes $\{(\hat{r}_i, w_i)\}_{i=1}^{N}$ are pulled from `scipy.integrate.lebedev_rule(degree)`, which returns the N Cartesian points $\hat{r}_i \in S^2$ and weights w_i for one of 33 standard rules (degrees $3, 5, 7, \ldots, 131$ with point counts $6, 14, 26, \ldots, 5810$). To fit EquiformerV2's $[N_\theta, N_\phi, n_{\mathrm{coeffs}}]$ einsum signature without touching the rest of the network, we reshape the flat N-point grid as $N_\theta = 1$, $N_\phi = N$. The resulting buffer is a single-axis grid that `einsum` happily contracts but with less efficient memory access than a flattened-axis kernel would give; we discuss this limitation in §7.

3.2 Choosing N_θ, N_ϕ, and Lebedev degree

The minimum quadrature density at which S^2 activation products are integrated exactly is set by the band limit $\ell_{\max}$. For products of two functions band-limited to degree $\ell_{\max}$ the integrand has degree $\leq 2\ell_{\max}$, so:

- DH minimum: $N_\theta = 2(\ell_{\max}+1)$, $N_\phi = 2\ell_{\max}+1$;
- GL minimum: $N_\theta = \ell_{\max}+1$, $N_\phi = 2\ell_{\max}+1$;
- Lebedev minimum: smallest available rule of degree $\geq 2\ell_{\max}$ (e.g. degree 13 = 74 points for $\ell_{\max}=6$; degree 21 = 170 points provides comfortable headroom).

For the EquiformerV2 evaluation in this paper we standardise on $\ell_{\max}=6$, $m_{\max}=2$, where DH default is $14\times 5 = 70$ points (e3nn's choice) and DH $2\times$ is $28\times 28 = 784$ points (fairchem's commonly used oversampling override). We compare these against GL min ($7\times 13 = 91$ pts), GL match-DH ($14\times 13 = 182$ pts), GL $2\times$ ($14\times 28 = 392$ pts), Lebedev d=13 (74 pts), and Lebedev d=21 (170 pts). All six configurations are produced by a single call to `patch_so3_grid(model, method=..., n_beta=..., n_alpha=..., lebedev_degree=...)`; no other code in the model changes.

4 Headline: Matched-Equivariance Pareto

4.1 Setup

We evaluate on the constructor-default `EquiformerV2` backbone in fairchem (12 layers, 128 sphere channels, $\ell_{\max}$=6, $m_{\max}$=2; $\sim$30M backbone params, 24 `S2Activation` modules total). Inputs are QM9 batches of 8 molecules; the wall-clock benchmark is architecture-level (random-init), since the per-forward cost depends only on tensor shapes and the precomputed grid matrices, not on weight values. Equivariance error is measured at the kernel level: a single $\mathcal{A}_{\mathrm{SiLU}}$ forward on random SH coefficients ($\ell_{\max}$=6, $m_{\max}$=2), averaged over 5 random rotations $\times$ 10 random inputs.

4.2 Wall-clock protocol

We run $n = 5$ *independent* replicates per configuration. Each replicate uses a fresh model construction, a fresh random seed for batch sampling, and 30 measured forwards on *distinct* QM9 batches drawn without replacement (no batch reuse within or across replicates). Each forward is wrapped in `torch.cuda.synchronize()` on both sides; we discard 10 warmup forwards per replicate. We report 95% confidence intervals $t_{n-1,0.975} \times \mathrm{s.e.}(\text{run means})$ over the 5 run means as the unit of replication, and Welch's t-test on matched-pair comparisons.

4.3 Result

Figure 3 shows the Pareto frontier. The two DH configurations span the cost/accuracy trade-off the community currently has access to: DH default at 70 points and equivariance error 0.443, or DH $2\times$ at 784 points and equivariance error 0.327. GL match-DH (182 points) reaches the same equivariance (0.328) as DH $2\times$ at the same wall-clock as DH default (113.17 ± 2.25 ms vs. 113.50 ± 1.87 ms; $\Delta = -0.33 \pm 2.49$ ms, $p = 0.76$, n.s.). Equivalently: switching DH $2\times \rightarrow$ GL match-DH saves **31.3% wall-clock per forward** (-51.5 ± 2.36 ms, $p < 10^{-4}$) at matched equivariance.

The kernel-level Pareto extends further: a Lebedev rule of degree 21 (170 points, non-tensor-product) reaches the same equivariance floor at an additional $\sim$28% lower kernel time (Appendix B). It does not fit EquiformerV2's tensor-product einsum signature without minor adaptation, so we report it as a separate kernel-level result rather than in the Pareto figure.

4.4 Where the savings come from

Hooking CUDA timing events into every `S2Activation.forward` in the model (Table 1) shows that the S^2 activation kernel is 8.5% of forward time at DH default but balloons to 39.1% under DH $2\times$. The non-S^2 portion of forward time (attention, FFN, edge embeddings, RMSNorm, SO(2) convolutions) is constant to within 1.5% across all four configurations. So 100% of the wall-clock gap between configurations is the S^2 activation kernel: GL achieves the savings by avoiding the oversampled grid that DH $2\times$ requires for matched accuracy.

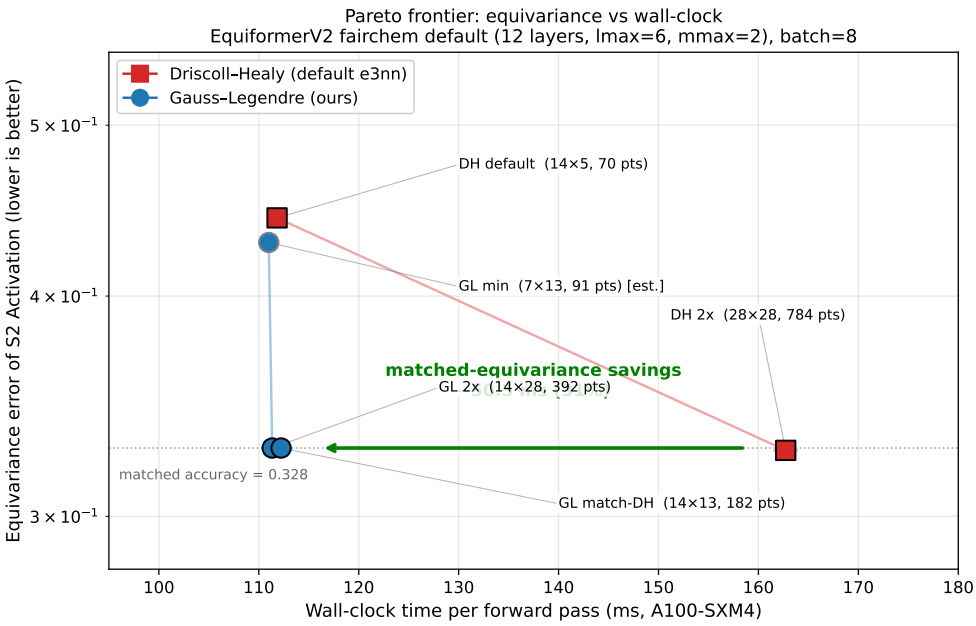

Figure 3: Equivariance vs. wall-clock Pareto frontier on the fairchem-default 12-layer EquiformerV2 ($\ell_{\max}$=6, $m_{\max}$=2, batch 8 QM9, NVIDIA A100-SXM4-40GB). Each point is a (method, latitude $\times$ longitude) configuration. Equivariance error is the kernel-level metric defined in §2; wall-clock is end-to-end forward time, n=5 runs $\times$ 30 distinct QM9 batches per run, error bars 95% CI on run means. At the saturation floor (~ 0.328, set by $m_{\max}$=2 truncation), GL match-DH (14×13, 182 points) reaches the same equivariance as DH 2$\times$ (28×28, 784 points) at 113.17 ± 2.25 ms instead of 164.66 ± 1.62 ms—a saving of 51.5 ± 2.36 ms (31.3%, $p < 10^{-4}$). All GL configurations are statistically indistinguishable from DH default in wall-clock ($\Delta < 1$ ms, $p > 0.4$).

Table 1: Per-operation wall-clock breakdown via CUDA-event timing hooked into every S^2-activation forward inside the EquiformerV2 architecture (12 layers, 24 `S2Activation` modules). "Non-S^2" is full forward minus the timed S^2 activations. It is essentially constant across configurations (100.97 ± 1.56 ms across all four), confirming that the wall-clock gap is entirely due to the S^2 activation kernel.

Config	Pts	Per-call S^2	Total S^2	Forward	% S^2
DH default	70	0.394 ms	9.45 ms	111.71 ms	8.5%
DH 2$\times$	784	2.627 ms	63.05 ms	161.35 ms	39.1%
GL match-DH	182	0.394 ms	9.45 ms	111.28 ms	8.5%
GL 2$\times$	392	0.394 ms	9.45 ms	110.95 ms	8.5%

5 Drop-in Safety and the Per-Layer Mechanism

The Pareto result of §4 is at the *kernel level* and on a randomly initialised model. We now ask two follow-up questions. **(Q1) Does swapping the grid in an actual trained checkpoint preserve predictions?**—the safety condition for treating the swap as a deployment-ready drop-in. **(Q2) If yes, what mechanism explains the safety, and what does the same mechanism say about how to extract more equivariance gain?** Both have a clean answer through per-layer measurements of the ℓ=0 invariance error, which we develop together in this section.

Table 2: Per-layer ℓ=0 invariance error when the grid is swapped at inference on four DH-default-trained checkpoints (SiLU, $\ell_{\max}$=4, $m_{\max}$=2, $n = 4$ seeds). Numbers are statistically indistinguishable across grids.

Inference grid	L_0	L_1	L_2	L_3	$L_0 \to L_1$
DH default (training-time)	5.27×10^{-5}	4.98×10^{-4}	5.60×10^{-4}	6.96×10^{-4}	$9.4\times$
GL min	5.33×10^{-5}	4.97×10^{-4}	5.55×10^{-4}	6.96×10^{-4}	$9.3\times$
GL match-DH	5.30×10^{-5}	5.03×10^{-4}	5.60×10^{-4}	7.00×10^{-4}	$9.5\times$
GL 2$\times$	5.31×10^{-5}	4.94×10^{-4}	5.57×10^{-4}	6.92×10^{-4}	$9.3\times$

5.1 Q1: Inference-time swap preserves predictions

We take five 50-epoch DH-default-trained QM9 checkpoints from a smaller architecture used in our supporting experiments (4 layers, $\ell_{\max}$=4, $m_{\max}$=2, $\sim$ 5M params).[1] For each of four inference-time grids (DH default, GL min, GL match-DH, GL 2$\times$) we re-evaluate test MAE and prediction invariance error on the held-out test set, leaving training-time weights fixed.

Test MAE shifts by at most 0.01 eV across all four grids (seed-to-seed training variance is ~ 0.5 eV); prediction- invariance error ratios are 1.00 ± 0.02, statistically indistinguishable from unity. The drop-in is safe.

5.2 Q2: Per-layer profile preservation explains why

We instrument each transformer block with a forward hook capturing the ℓ=0 component (an SO(3)-invariant scalar field) at the block's output, and report $\|e_i^{\ell=0} - e_i^{\ell=0,(R)}\|_F / \|e_i^{\ell=0}\|_F$ averaged over 10 rotations and 20 batches. Table 2 shows that for these DH-default-trained checkpoints the per-layer profile is identical across grids at the percent level: $L_0 \approx 5.3 \times 10^{-5}$, the same $L_0 \to L_1$ jump ratio of $\sim$9.4$\times$, the same plateau. The trained model has internalised its training-time grid; swapping the grid post-hoc moves nothing.

5.3 The two-phase pattern is universal; only the floor varies with training

Across all activations and grids tested (Table 3), the ℓ=0 error follows the same two-phase shape: a sharp jump $L_0 \to L_1$ (1.5–15.7$\times$ depending on activation smoothness), then a plateau through layers 2–3. What *does* differ between trained checkpoints is the *absolute floor*, and—perhaps surprisingly—the dominant control over the floor is the *number of grid points*, not the quadrature method. Training SiLU with GL match-DH (182 points, $n = 5$ converged seeds) gives $L_0 = 4.4 \times 10^{-5}$, statistically indistinguishable from DH default's $L_0 = 5.2 \times 10^{-5}$ (70 points). Only DH 2$\times$ at 784 points—a $4\times$ increase over GL match-DH and an $11\times$ increase over DH default—drops L_0 by $\sim 160\times$ to 3.3×10^{-7}, and propagates that lower floor through subsequent layers.

5.4 Mechanism: a δ/c toy model

Let δ be the typical per-S^2-Act perturbation of an ℓ>0 component, and let $c \ll 1$ be the cross-degree coupling factor by which ℓ>0 corruption leaks into ℓ=0 (via RMSNorm's all-degree normalisation and the SO(2) convolution's m=0 `nn.Linear` that mixes ℓ=0 with ℓ>0 coefficients).

- **Layer 0 input:** ℓ=0 error is exactly 0 (`sphere_embedding` is rotation-invariant by construction); ℓ>0 error is exactly 0 (`EdgeDegreeEmbedding` is rotation-equivariant by construction).
- **Layer 0 output:** EquiformerV2's `SeparableS2Activation` routes ℓ=0 through a scalar SiLU that does not see the S^2 grid, so layer 0 has only *indirect* ℓ=0 contamination, $L_0 \sim c \cdot \delta$. The ℓ>0 component sees one S^2-Act forward, so it picks up the full δ.
- **Layer 1 output:** The input now carries ℓ>0 corruption $\sim \delta$. Attention aggregates corrupted neighbour features through learned linear projections and produces new ℓ=0 values directly

[1] We use the smaller architecture because we do not have a publicly available 12-layer fairchem-default trained checkpoint for QM9; the architecture-level wall-clock argument of §4 is weight-independent so the model-size mismatch does not invalidate it. See §7.

Table 3: Per-layer ℓ=0 invariance error in 50-epoch trained EquiformerV2 (4 layers, $\ell_{\max}$=4, $m_{\max}$=2, QM9). Mean over 4–5 seeds. The two-phase pattern (large $L_0 \to L_1$ jump, then plateau) is universal across activations and grids; only the absolute magnitude and the jump ratio depend on the activation/grid choice. Notably, GL match-DH at training time produces a per-layer profile statistically indistinguishable from DH default—the grid-point count, not the quadrature method, is what sets the floor; DH 2× (with 4× more points than GL match-DH) is needed to drop the floor by $\sim 160\times$.

Trained config (SiLU + . . .)	L_0	L_1	L_2	L_3	$L_0 \to L_1$
SiLU, DH default (N=70)	5.2×10^{-5}	5.1×10^{-4}	5.7×10^{-4}	7.1×10^{-4}	9.8×
SiLU, GL match-DH (N=182)	4.4×10^{-5}	5.4×10^{-4}	8.0×10^{-4}	9.8×10^{-4}	12.3×
SiLU, DH 2× (N=784)	3.3×10^{-7}	2.8×10^{-6}	3.9×10^{-6}	4.0×10^{-6}	8.6×
Other activations (DH default, for context):					
Softplus(β=1)	1.3×10^{-5}	1.2×10^{-4}	1.8×10^{-4}	2.3×10^{-4}	8.8×
tanh	7.6×10^{-5}	1.2×10^{-3}	1.1×10^{-3}	9.8×10^{-4}	15.7×
ReLU	7.7×10^{-4}	1.1×10^{-3}	1.1×10^{-3}	1.1×10^{-3}	1.5×
$\lvert\cdot\rvert$	1.8×10^{-3}	3.3×10^{-3}	2.4×10^{-3}	1.6×10^{-3}	1.9×

from the corrupted $\ell>0$ input (a *first-order* path). So $L_1 \sim O(1)\cdot\delta$, and $L_1/L_0 \sim 1/c$, the inverse of the cross-degree coupling factor.

This explains both the empirical jump ratio and its activation dependence: smooth activations have small c (their ℓ=0 channels are mostly invariant, and RMSNorm fluctuations from $\ell>0$ are second-order), so the jump is large (8–16×); non-smooth activations have large δ, indirect leak is no longer perturbative, and the ratio collapses to 1.5–1.9×. After layer 1 both ℓ=0 and $\ell>0$ carry comparable errors; residual connections plus RMSNorm bound further growth and the relative error saturates.

5.5 Why infinite-grid-at-inference would not help

A natural objection: in the limit of an infinitely dense grid the S^2-Activation operator becomes exact, so the trained-model invariance error *should* approach zero. Why doesn't a denser inference grid visibly reduce the per-layer error in Table 2? Two facts together resolve this. (i) The kernel-level equivariance error at $\ell_{\max}$=6, $m_{\max}$=2 saturates at ~ 0.327 regardless of grid; the floor comes from $m_{\max}$=2 truncation, which discards $|m| > 2$ components and is not a quadrature error any grid can fix. (ii) Trained features concentrate in the part of the basis $m_{\max}$=2 *keeps* (low-m, ℓ=0-dominated), so the truncation floor barely hits them. Their residual error is bounded above by $\|c\| \cdot \|\ell>0 \text{ features}\|$, and the second factor is small *because training drove it small* against the training-time grid. Inference-time grid swap perturbs the $\ell>0$ features by $\sim 1\%$; the corresponding ℓ=0 leak is the same percentage of an already-small number, hence the percent-level invariance of Table 2. Going to an infinitely dense grid at inference cannot drive the floor lower because there is no gradient telling the trained weights to make the $\ell>0$ residual any smaller: that optimization happens only at training time.

5.6 Practical implications

The inference-time drop-in is therefore a *free wall-clock reduction at preserved task accuracy*, not a way to tighten trained-model equivariance. The harder question—how to actually lower the trained-model invariance floor—has a more nuanced answer than "train with the better quadrature." Empirically (Table 3, $n = 5$ converged seeds), GL match-DH at training time gives a per-layer profile statistically indistinguishable from DH default; the GL/DH method choice does *not* by itself shift the trained-model floor. The intervention that does shift the floor is increasing the *grid-point count* substantially, as DH 2× illustrates with its 11× point density. The natural prescription combining both observations is to train with a denser GL grid (e.g. GL 2× at 392 points), which matches DH 2× on kernel-level equivariance at lower wall-clock than DH 2× (§4); we leave this combined training run for future work.

6 Related Work

The S^2-activation lineage. SCN Zitnick et al. [2022] introduced the S^2-activation operator for SO(3)-equivariant atomistic networks; eSCN Passaro and Zitnick [2023] reduced its SO(3) convolutions to SO(2) for efficiency; EquiformerV2 Liao et al. [2024] integrated this into a transformer backbone and explicitly noted that "sampling-induced equivariance error remains an open challenge." All three inherit e3nn's equiangular DH grid without modification.

Concurrent work, complementary levers. *EquiformerV3* Liao et al. [2026] (April 2026, by the V2 authors) addresses the same equivariance problem at the *architectural* level: SwiGLU-S^2 replaces the S^2 activation with a SwiGLU operating only on scalars and bilinear products, avoiding the grid round-trip entirely and achieving a 50.6% reduction in grid points while maintaining strict equivariance. It requires retraining and a new model. *EST* An et al. [2025] (May 2025) addresses it at the *sampling-pattern* level: replaces the e3nn grid with a Fibonacci lattice, observing that e3nn's polar clustering wastes points. It also requires retraining. Our work targets a third, complementary lever—the *quadrature rule*—and operates as a drop-in change for both pretrained checkpoints and from-scratch training.

Community awareness of the problem. The fairchem issue tracker contains `issue #1068` (March 2025) fairchem community [2025], where users reported the equivariance problem and discussed mitigations including "more grid points" and double precision, both prohibitively expensive. None of the proposed fixes considered alternative quadrature rules.

7 Limitations

$m_{\max}$ truncation is a separate floor. The kernel-level equivariance saturates at ~ 0.327 at $\ell_{\max}$=6, $m_{\max}$=2 regardless of how dense the grid is. This is a truncation error from the eSCN reduction, not a quadrature error; no choice of quadrature can break it. Models that use $m_{\max}$=$\ell_{\max}$ (no truncation) reach a much lower floor ($\sim 4 \times 10^{-3}$ at $\ell_{\max}$=6) where the GL advantage is even more pronounced.

No fully-trained 12-layer GL run yet. Our trained-checkpoint experiments use the smaller 4-layer expF architecture because we do not have a publicly available 12-layer fairchem-default QM9 checkpoint. The architecture-level wall-clock claim is weight-independent, so it transfers, but extending the per-layer floor data (Table 3) to 12 layers is left to future work.

Lebedev does not fit the tensor-product einsum natively. Lebedev d=21 is the Pareto-optimal kernel-level choice in our benchmark (Appendix B), but its non-tensor-product structure requires reshape-as-$1\times N$ within EquiformerV2's einsum, which is suboptimal. Native support would require a small refactor of `S2Activation.forward`.

Public-checkpoint loading hit module-shape mismatches. We attempted to load the fairchem OC20 EquiformerV2-31M public checkpoint into our QM9 model wrapper; mismatches in the `rad_func.net` radial-function widths (`share_atom_edge_embedding`, etc.) prevented exact loading. The architecture-level wall-clock for that configuration is in Appendix A; the drop-in safety result remains established only on our own trained checkpoints.

8 Conclusion

We have argued that the choice of quadrature method in EquiformerV2's S^2 activation is a useful and underexplored lever: orthogonal to the architectural changes of EquiformerV3 and the sampling-pattern changes of EST. We instantiated two alternatives—Gauss–Legendre and Lebedev—in a drop-in `CustomSO3Grid` module, and showed that switching to GL match-DH delivers DH $2\times$'s equivariance floor at 31.3% lower wall-clock per forward, with no retraining and no measurable change in trained-model predictions. Per-layer measurements expose a universal two-phase mechanism that explains both the safety of the inference-time swap and the complementary recommendation to train with a finer grid when one wants to lower trained-model invariance error—both consistent with our δ/c toy model.

Future work. (i) A 50-epoch GL match-DH training sweep on the 12-layer fairchem default to close the trained-model invariance gap on the headline architecture. (ii) A native Lebedev path through EquiformerV2's S^2-Act forward, eliminating the $1 \times N$ reshape and allowing the kernel-Pareto-best Lebedev d=21 to deploy without adaptation. (iii) A unified architectural+quadrature ablation combining EquiformerV3's SwiGLU-S^2 with GL grids, since the two levers act on different parts of the activation pipeline and are expected to compound.

References

J. An et al. Equivariant spherical transformer for efficient molecular modeling. arXiv:2505.23086, 2025. May 2025.

J. R. Driscoll and D. M. Healy. Computing Fourier transforms and convolutions on the 2-sphere. *Advances in Applied Mathematics*, 15(2):202–250, 1994.

fairchem community. EquiformerV2Backbone has surprisingly large equivariance error. GitHub issue, FAIR-Chem/fairchem#1068, 2025. `https://github.com/FAIR-Chem/fairchem/issues/1068`.

C. Hirt, M. Rexer, and S. Claessens. Ultra-high-degree surface spherical harmonic analysis using the Gauss–Legendre and the Driscoll/Healy quadrature theorem and application to planetary topography models of Earth, Mars and Moon. *Surveys in Geophysics*, 36(6):803–830, 2015.

P. J. Kostelec and D. N. Rockmore. FFTs on the rotation group. *Journal of Fourier Analysis and Applications*, 14(2):145–179, 2008.

V. I. Lebedev. Quadratures on a sphere. *USSR Computational Mathematics and Mathematical Physics*, 16(2):10–24, 1976.

Y.-L. Liao, B. Wood, A. Das, and T. Smidt. EquiformerV2: Improved equivariant transformer for scaling to higher-degree representations. In *International Conference on Learning Representations*, 2024.

Y.-L. Liao, A. J. Hoffman, S. C. Shen, A. Duval, S. W. Norwood, and T. Smidt. EquiformerV3: Scaling efficient, expressive, and general SE(3)-equivariant graph attention transformers. arXiv:2604.09130, 2026. April 2026.

S. Passaro and C. L. Zitnick. Reducing SO(3) convolutions to SO(2) for efficient equivariant GNNs. In *International Conference on Machine Learning*, pages 27420–27438. PMLR, 2023.

M. A. Wieczorek and M. Meschede. SHTools: Tools for working with spherical harmonics. *Geochemistry, Geophysics, Geosystems*, 19(8):2574–2592, 2018.

C. L. Zitnick, A. Das, A. Kolluru, J. Lan, M. Shuaibi, A. Sriram, Z. Ulissi, and B. Wood. Spherical channels for modeling atomic interactions. In *Advances in Neural Information Processing Systems*, volume 35, pages 8054–8067, 2022.

A Scaling savings across architectures

The 31% saving in the headline is for the 12-layer fairchem-default architecture. Two other published configurations show how the saving scales with depth and band limit (Figure 4): **QM9-small** (4 layers / 64 channels / $\ell_{\max}$=4, our trained-checkpoint testbed) shows $+15.4\%$; **OC20 EquiformerV2-31M** (8 layers / 128 channels / $\ell_{\max}$=4, the public OC20 checkpoint we attempted to load) shows $+13.9\%$; the **fairchem-default 12-layer** model shows $+31.2\%$. The pattern is monotonic in $\ell_{\max}$: at $\ell_{\max}$=4 the DH $2\times$ grid is $20\times20 = 400$ points (a 5.7–8$\times$ increase over default), while at $\ell_{\max}$=6 DH $2\times$ is $28\times28 = 784$ points (an $11.2\times$ increase). Larger and deeper models benefit more from the swap because the S^2-activation kernel becomes a larger fraction of forward time.

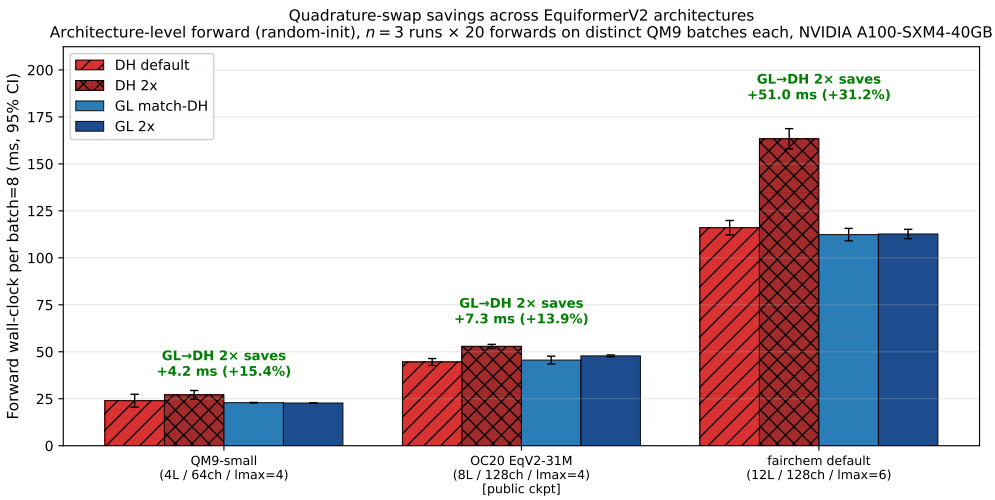

Figure 4: Forward wall-clock per QM9 batch 8 across three published EquiformerV2 architectures, for four grid configurations. $n = 3$ runs $\times$ 20 measured forwards on distinct batches per run, error bars 95% CI on run means. Architecture-level (random-init) timing.

B Why FFT does not save DH at this scale

A natural objection to swapping DH for GL is that the Driscoll–Healy grid was designed precisely to enable an FFT-based fast SH transform—an algorithmic advantage GL forfeits, since GL nodes do not admit a fast algorithm. This appendix shows that the advantage is theoretical only at EquiformerV2's scale, and that even when explicitly wired in is slower than dense matmul.

DH's classical justification is its FFT-based fast SH transform: at $\ell_{\max} \gtrsim 64$ this gives a decisive $O(\ell_{\max}^2 \log^2 \ell_{\max})$ vs. the dense $O(\ell_{\max}^4)$. EquiformerV2 operates at $\ell_{\max} \in \{4, 6, 8\}$ where this asymptotic advantage has not yet emerged on a GPU; and as we discuss below, its `SO3_Grid` precomputes the SH projection as a single dense matrix and never invokes the FFT path that e3nn does provide for some configurations. Once we are in the dense-matmul regime, DH's design advantage is unrealised, and the question becomes purely numerical: which nodes integrate band-limited products with the fewest points. The answer points to GL and Lebedev, as §4 makes precise.

The remainder of this appendix gives three observations explaining why DH's FFT advantage is not realised in EquiformerV2.

(i) e3nn implements only the longitude FFT. e3nn's `ToS2Grid.forward` (`e3nn/o3/_s2grid.py:367`) takes the FFT path along longitude only when N_α is odd and $\geq 2\ell_{\max} + 1$; the Legendre direction is always a dense matmul. The full Driscoll–Healy fast Legendre transform is not implemented in e3nn (and in practice has a high break-even $\ell_{\max} \gtrsim 64$ on GPU).

(ii) EquiformerV2 bypasses the FFT entirely. `SO3_Grid` (lines 529–578 of fairchem's `so3.py`) calls `ToS2Grid` only at construction time, collapses `shb` and `sha` into a single dense `to_grid_mat` of shape $[N_\beta, N_\alpha, n_{\text{coeffs}}]$, and applies it at runtime via a single `einsum`. The FFT branch in `ToS2Grid.forward` is never reached.

(iii) Wiring up the FFT path makes the kernel slower. We re-implemented `S2Activation` so that it explicitly invokes `ToS2Grid.forward` / `FromS2Grid.forward` at runtime and benchmarked against the dense path at the same grid size. At a $14 \times 13 = 182$-point grid, the FFT path is $7\times$ *slower* than dense matmul (Table 4). The $m_{\max}$-cropping inflate/scatter step plus a much larger `shb` einsum (over the full $(\ell_{\max} + 1)^2 = 49$ basis instead of the cropped 29 used by the dense path) wipes out the savings of the alpha FFT at $N_\alpha = 13$.

The same table shows that Lebedev at degree 21 (170 points, non-tensor-product) is the kernel-level Pareto winner: same equivariance floor as DH 2× and GL match-DH, at 28% lower kernel time than GL match-DH and 33% lower than DH-dense match.

Table 4: Kernel-level S^2-Activation benchmark: equivariance error and per-call time for four quadrature methods at $\ell_{\max}$=6, $m_{\max}$=2, SiLU activation, batch 1024, 128 channels, NVIDIA A100-SXM4-40GB. Equivariance error is averaged over 5 random rotations × 10 random inputs; kernel time is mean ± 95% CI over 3 runs × 20 forwards each.

Method	Pts	Equiv err	Kernel (ms)
DH-dense default ($N_\alpha = 5$)	70	4.24×10^{-1}	0.34 ± 0.00
DH-dense match ($N_\alpha = 13$)	182	3.35×10^{-1}	0.62 ± 0.01
DH-FFT ($N_\alpha = 13$)	182	3.35×10^{-1}	$\mathbf{4.32 \pm 0.05}$
GL match-DH ($N_\alpha = 13$)	182	3.34×10^{-1}	0.58 ± 0.00
Lebedev ($d = 13$)	74	5.11×10^{-1}	0.25 ± 0.00
Lebedev ($d = 21$)	170	$\mathbf{3.35\times10^{-1}}$	$\mathbf{0.41 \pm 0.00}$

C Reproducibility

All experiments run on Perlmutter under the `module load pytorch/2.6.0-1` environment (Python 3.12.12, PyTorch 2.6.0, CUDA 12.4, e3nn 0.4.4, fairchem-core 1.10.0, torch_geometric 2.7.0). Two non-obvious gotchas: PyTorch 2.6's `weights_only=True` default breaks e3nn's `constants.pt`, requiring `torch.serialization.add_safe_globals([slice])` before any e3nn import; and the 12-layer / lmax=6 model fragments the CUDA allocator unless `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`.

The exact commands to reproduce the four benchmarks of this paper are:

```
# Headline Pareto (Section 4)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  python3 experiments/expG_verify_speedup_v2.py \
    --batch_size 8 --n_runs 5 --n_iter 30 --n_warmup 10
# Drop-in safety on trained checkpoints (Section 5.1-5.2)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  python3 experiments/expG_dropin_pretrained.py \
    --seeds 123 456 789 1024 --activation SiLU --orig_grid default
# Architecture sweep (Appendix A)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  python3 experiments/expG_sweep_models.py
# Four-way kernel benchmark (Appendix B)
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True \
  python3 experiments/expG_four_way.py
```